Lliurament (al Moodle):

- Un **PDF** amb les captures de Postman de totes les proves del **checklist** (apartat 9), i una explicació del `package.json`: què fa cada script i cada dependència.
- El fitxer `app.js` degudament comentat.
- La **col·lecció de Postman** exportada (*Export* → fitxer `.json`).

⚠️ El `package.json` no admet comentaris: si hi escriviu `//`, npm deixarà de funcionar. Expliqueu-lo al PDF.

Enunciat: Pràctica CRUD amb Express – Gestió de Productes

Objectiu

Crear una **API REST** per gestionar un catàleg de productes amb **Express**, implementant totes les operacions **CRUD** (Create, Read, Update, Delete). Les dades es guardaran en memòria (en un array), sense base de dades.

Planificació: 2 sessions

Sessió	Què heu de tenir fet en acabar
Sessió 1	Projecte configurat, dades inicials, els dos GET, el POST (encara sense validacions) i la ruta 404. És el que hem vist a classe.
Sessió 2	PUT i DELETE (cal investigar-los, vegeu l'apartat 8), validacions, gestor d'errors, proves amb Postman i lliurament.

1. Model Producte

Cada producte té les propietats següents:

```
{
    id: number,          // Identificador únic (el genera el servidor)
    nom: string,         // Nom del producte
    preu: number,        // Preu en euros
    categoria: string,   // Categoria del producte
    stock: number        // Quantitat disponible
}
```

L'`id` **no** l'envia mai el client: el genera el servidor en crear el producte, i no es pot modificar.

2. Endpoints a implementar

Mètode	Ruta	Què fa	Codis de resposta
GET	`/api/productes`	Llista tots els productes	200
GET	`/api/productes/:id`	Retorna un producte	200 · 400 id no vàlid · 404
POST	`/api/productes`	Crea un producte	201 · 400 dades invàlides
PUT	`/api/productes/:id`	Actualitza un producte	200 · 400 · 404
DELETE	`/api/productes/:id`	Elimina un producte	200 · 400 id no vàlid · 404

GET /api/productes

- Retorna tots els productes.
- Si no n'hi ha cap, retorna un array buit (amb codi 200, no és un error).

GET /api/productes/:id

- Retorna el producte amb aquest id.
- Si l'id no és un número (per exemple `/api/productes/abc`), retorna **400**.
- Si l'id és vàlid però no existeix cap producte amb aquest id, retorna **404**.

POST /api/productes

- El cos de la petició ha d'incloure: `nom`, `preu`, `categoria` i `stock`.
- Les dades s'han de validar (apartat 3). Si no són vàlides, retorna **400**.
- L'id s'ha de generar automàticament i ha de ser **únic**, fins i tot després d'esborrar productes (vegeu la pregunta de l'apartat 9).
- Retorna el producte creat, amb codi **201**.

PUT /api/productes/:id

- El cos de la petició ha d'incloure **tots** els camps (`nom`, `preu`, `categoria`, `stock`), amb les mateixes validacions que el POST.
- L'`id` no es pot modificar: si el cos inclou un `id`, s'ignora.
- Si el producte no existeix, retorna **404**.
- Retorna el producte actualitzat, amb codi **200**.

DELETE /api/productes/:id

- Elimina el producte.
- Si el producte no existeix, retorna **404**.
- Retorna un missatge de confirmació i el producte eliminat, amb codi **200**.

3. Validacions requerides

S'apliquen al POST i al PUT. Cal comprovar tots els camps i retornar tots els errors alhora:

Camp	Tipus	Condició
`nom`	`string`	No pot estar buit. Un text amb només espais (" ") també compta com a buit.
`preu`	`number`	Ha de ser un número positiu (més gran que 0).
`categoria`	`string`	No pot estar buida (tampoc només amb espais).
`stock`	`number`	Ha de ser un número enter positiu o zero.

⚠️ **Compte amb els tipus:** `"preu": 29.99` és vàlid, però `"preu": "29.99"` (entre cometes, és un text) **no** ho és.

4. Estructura de les respostes

Totes les respostes de l'API han de seguir aquest format (el mateix que hem fet servir a classe):

Èxit

```
{
    "estat": "success",
    "missatge": "Producte creat correctament",   // opcional en els GET
     "data": { ... }                                  // producte o array de
productes
}
```

Error

```
{
    "estat": "error",
    "missatge": "Producte no trobat"
}
```

Error de validació (400)

```
{
    "estat": "error",
    "missatge": "Dades invàlides",
    "detalls": [
        "El nom no pot estar buit",
        "El preu ha de ser un número positiu"
    ]
}
```

5. Codis HTTP a utilitzar

Codi	Quan
200	Operació correcta (GET, PUT, DELETE).
201	Recurs creat correctament (POST).
400	Error en les dades enviades: validació fallida, id no numèric o JSON mal format.
404	Recurs no trobat (producte inexistent o ruta inexistent).
500	Error intern del servidor (qualsevol error inesperat, recollit pel gestor d'errors).

6. Gestió d'errors

Ruta 404

Qualsevol ruta que no existeixi ha de respondre en JSON, amb codi 404 i el camí que s'ha demanat:

```
{
    "estat": "error",
    "missatge": "Ruta no trobada",
    "ruta": "/api/clients"
}
```

Gestor d'errors

Cal un middleware d'errors (`app.use((err, req, res, next) => { ... })`) que:

- Respongui **400** en JSON quan el client envia un JSON mal format.
- Respongui **500** en JSON davant de qualsevol altre error, **sense** enviar al client els detalls interns (rutes de fitxers, traça de l'error...).

7. Ordre del fitxer app.js

A Express l'ordre importa. El vostre app.js ha de seguir aquesta estructura:

```
1. Imports i configuració          require, express(), PORT
2. Middleware                      app.use(express.json())
3. Dades inicials                  let productes = [...]
4. Rutes                           app.get / app.post / app.put / app.delete
5. Ruta 404                        app.use((req, res) => ...)
6. Gestor d'errors                 app.use((err, req, res, next) => ...)
7. Engegar el servidor             app.listen(...)
```

Dades inicials

```
let productes = [
    {
        id: 1,
        nom: "Portàtil Gaming",
        preu: 999.99,
        categoria: "Informàtica",
        stock: 10
    },
    {
        id: 2,
        nom: "Ratolí sense fil",
        preu: 29.99,
        categoria: "Accessoris",
        stock: 25
    }
];
```

8. Per investigar

El PUT i el DELETE no els hem fet a classe: us toca investigar-los. Aquestes eines us seran útils:

Eina	Per a què us pot servir
`app.put()` · `app.delete()`	Definir rutes per a aquests mètodes (funcionen com app.get i app.post).
`array.findIndex()`	Trobar la posició d'un producte dins l'array.
`array.splice()` o `array.filter()`	Eliminar un element de l'array.
`typeof`	Comprovar si un valor és un number o un string.
`Number.isInteger()`	Comprovar si un número és enter (per a l'stock).
`Number.isNaN()`	Detectar un id que no és un número (parseInt("abc") és NaN).
`string.trim()`	Treure els espais del principi i del final d'un text.

Documentació: expressjs.com (Express 5) i developer.mozilla.org (JavaScript).

9. Checklist de proves amb Postman

Feu totes aquestes proves, guardeu-les a la col·lecció de Postman i incloeu-ne la captura al PDF:

#	Petició	Resultat esperat
1	`GET /api/productes`	200 + llista de productes
2	`GET /api/productes/1`	200 + el producte 1
3	`GET /api/productes/999`	404
4	`GET /api/productes/abc`	400
5	`POST /api/productes` amb totes les dades correctes	201 + producte amb id nou
6	`POST` amb el nom buit o només espais	400 + detalls
7	`POST` amb el preu negatiu o entre cometes	400 + detalls
8	`POST` amb un stock decimal (`2.5`)	400 + detalls
9	`POST` amb diversos camps incorrectes alhora	400 + tots els errors
10	`POST` amb un JSON mal format (sense la `}` final)	400 en JSON
11	`PUT /api/productes/1` amb dades correctes	200 + producte actualitzat
12	`PUT /api/productes/999`	404
13	`PUT /api/productes/1` amb dades incorrectes	400 + detalls
14	`DELETE /api/productes/2`	200 + confirmació
15	`GET /api/productes/2` (després d'esborrar-lo)	404
16	`DELETE /api/productes/999`	404
17	`GET /api/clients` (ruta inexistent)	404 + ruta

> 🤔 **Pregunta de reflexió (responeu-la al PDF)**
>
> Engegueu el servidor de nou, esborreu el producte 1 i després creeu dos productes nous. Quins ids han rebut? Es repeteix algun id? Expliqueu com genereu l'id i per què és (o no és) segur.

Extres opcionals

Per a qui acabi abans. Sumen nota, però no són obligatoris:

- `GET /api/productes?categoria=Informàtica`: filtrar per categoria (investigueu `req.query`).
- `PATCH /api/productes/:id`: actualitzar només els camps que s'envien.
- `GET /api/productes?ordre=preu`: retornar els productes ordenats per preu.

Criteris d'avaluació

Criteri	Pes
Operacions CRUD implementades i funcionant (els 5 endpoints)	40%
Validacions correctes (tipus, buits, enters, missatges de detall)	20%
Gestió d'errors i ús correcte dels codis HTTP (404, gestor d'errors, JSON invàlid)	20%
Codi net, ben organitzat (ordre de l'app.js) i ben comentat	10%
Proves amb Postman (checklist complet, col·lecció i pregunta de reflexió)	10%
Extres opcionals	*fins a +1 punt*